Зміст

1 Синтаксис Python. Типи Даних 2

1.1 Змінні та Ідентифікатори . . . 2

1.1.1 Правила побудови ідентифікаторів . . . 2

1.2 Зарезервовані слова Python . . . 2

1.2.1 Приклад помилки . . . 3

1.3 Оператори та Вирази . . . 3

1.3.1 Правила форматування коду . . . 4

1.4 Основні Типи Даних і Операції . . . 4

1.5 Приклади. Виділення цифр числа . . . 6

1.5.1 Двоцифрове число . . . 6

1.5.2 Трицифрове число . . . 6

1.6 Вбудовані функції для роботи з числами . . . 7

1.6.1 Функція `abs(a)` . . . 7

1.6.2 Функція `divmod(a, b)` . . . 7

1.6.3 Функція `hex(x)` . . . 7

1.6.4 Функція `oct(x)` . . . 7

1.6.5 Функція `round(x)` . . . 8

1.6.6 Функція `round(x, n)` . . . 8

1.7 Оператори інкременту та особливості `float` . . . 8

2 Оператор циклу `for` та функція `range()` 11

2.1 Генератор послідовностей `range()` . . . 11

2.2 Базові алгоритми з циклом `for` . . . 12

2.3 Управління циклом: `break`, `continue` та `else` . . . 16

Синтаксис Python. Типи Даних

1

1.1 Змінні та Ідентифікатори

Змінна - це

ім’я (ідентифікатор), яке може набувати певного значення.

У Python змінні не потрібно оголошувати, значення можна надати одразу за допомогою символу =.

Приклади присвоєння

```
x = 4
s1 = "Hello"
_x = True
x2 = 23.46
summa1 = summa2 = k = 0
```

Ідентифікатор - це

унікальна ознака об’єкта, що дозволяє розрізняти його з іншими.

Правила побудови ідентифікаторів

- ☑ Використовуються лише латинські літери, цифри та знак підкреслення (_).
- ☑ Ідентифікатор повинен починатися з літери або символу підкреслення.
- ☑ Регістр літер має значення (`X` ≠ `x`).
- ☑ Неприпустимі пробіли, точки, спеціальні символи та літери кирилиці.
- ☑ Не можна застосовувати спеціальні символи `$`, `%`, `@`.
- ☑ Заборонено використовувати зарезервовані слова Python (`if`, `for`, `while`, `class`, `def` тощо).

1.2 Зарезервовані слова Python

У Python існує набір **ключових слів**, які не можна використовувати як ідентифікатори (назви змінних, функцій, класів тощо). Ці слова мають спеціальне значення

у синтаксисі мови.

```
False   class     finally  is       return
None    continue  for      lambda   try
True    def       from     nonlocal while
and     del       global   not      with
as      elif      if       or       yield
assert  else      import   pass     break
```

Використання будь-якого з цих слів як назви змінної або функції призведе до помилки.

Приклад помилки

```
>>> class = 5
File "<stdin>", line 1
class = 5
^
SyntaxError: invalid syntax
```

1.3 Оператори та Вирази

Вираз — це

фрагмент мови програмування, що представляє спосіб обчислення деякого значення.

Операнд — це

частина виразу, над яким виконуються операції.

Оператор — це

дія, яка може бути виконана над одним або декількома операндами.

Приклади синтаксису

```
X=4+5 % Присвоєння
Print(x)
S="Hello "+"World"
Print(s)
S="Helo"+\
"World" % Перенесення довгих операторів
```

Правила форматування коду

- ☑ Кожен оператор завершується символом нового рядка.
- ☑ Відступи застосовуються для позначення різних блоків коду (тіла функцій, умовні висловлювання, цикли).
- ☑ Пробіли неприпустимі перед операторами, але необхідні перед вкладеними блоками (`unexpected indent`).
- ☑ Якщо потрібно розмістити кілька операторів в одному рядку, їх розділяють крапкою з комою (`;`).
- ☑ Символ # означає коментар.
- ☑ Багаторядковий коментар укладають у три подвійні лапки в `py` файлі.

1.4 Основні Типи Даних і Операції

Вбудовані типи даних:

- ☑ Числа: `1234`, `3.14`, `3+4j`.
- ☑ Рядки: `'spam'`, `"guido's"`.
- ☑ Списки: `[1, [2, 'three'], 4]`.
- ☑ Словники: `{'food': 'spam', 'taste': 'yum'}`.
- ☑ Кортежі: `(1, 'span', 4, 'U')`.
- ☑ Файли: myfile = open(’eggs’, ‘r’)
- ☑ Множини: set(’abc’). ’a’,’b’, ’c’

Вбудовані числові типи

- ☑ цілі числа (числа без дробової частини), (3, 3465, 45, 42, -4) (з необмеженою точністю)
- ☑ дійсні числа (грубо кажучи, числа з десятковою точкою з плаваючою точкою) (3,15 5,68)

- ☑ 8-річні, починаються нуль про 0o77 = 63 0o10 = 8
- ☑ 16-річні, починаються нуль x 0xFF=255 0xff=255 • - двійкові, починаються нуль b 0b1111 = 15 0b10 = 2
- ☑ - комплексні числа з уявною частиною, (3+4j), (3+4J)(з допомогою великої та маленької) літер)

Символ множення перед j не вказується.

```
>>>z1=2+3j
>>>z2=4+8J
>>>print(z1, z2)
(2+3j) (4+8j)
```

Комплексні числа можуть бути створені за допомогою функції complex(реч _ частина, уявна _ частина)

```
>>>z3=complex(34,2)
>>>print(z3)
(34+2j)
```

Операції з числовими типами

Оператор	Опис	Приклад	Результат
+	Додавання	5+7	12
-	Віднімання	55-10	45
*	Множення	4*6	24
/	Ділення	7 / 2	3.5
//	Цілочисельне ділення	7 // 2	3
%	Остача від ділення	7 % 3	1
**	Піднесення до степеня	3 ** 4	81

Результатом операції буде новий об’єкт.

2 ● ● 100#2100

1267650600228229401496703205376

Зверніть увагу на результат останньої операції: у Python 3 цілі числа автоматично забезпечують **необмежену** точність. Наприклад, ви можете спробувати обчислити 2 ступенем 1 000 000 (але навряд чи варто це робити, оскільки на екран буде виведено число довжиною понад 300 000 знаків, що може зайняти тривалий

час!).

> len(str(2 ** 1000000))

Скільки цифр у дійсно великому числі

301030

1.5 Приклади. Виділення цифр числа

Двоцифрове число

Нехай маємо число `n = 47`. Щоб виділити окремі цифри:

Двоцифрове число

```
n = 47
d1 = n // 10    # перша цифра (десятки)
d2 = n % 10     # друга цифра (одиниці)
print("Десятки =", d1, "Одиниці =", d2)
```

Результат:

```
Десятки = 4 Одиниці = 7
```

Трицифрове число

Нехай маємо число `m = 583`. Щоб виділити окремі цифри:

Трицифрове число

```
m = 583
sot = m // 100          # сотні
des = (m // 10) % 10    # десятки
od = m % 10             # одиниці
print("Сотні =", sot, "Десятки =", des, "Одиниці =", od)
```

Результат:

```
Сотні = 5 Десятки = 8 Одиниці = 3
```

Пріоритет операторів - порядок виконання тут

1.6 Вбудовані функції для роботи з числами

Python надає низку **вбудованих функцій**, які можна застосовувати до числових типів.

Функція `abs(a)`

Повертає модуль числа.

Приклад

```
x = -12
print(abs(x))    # 12
```

Функція `divmod(a, b)`

Виконує ділення `a` на `b` та повертає пару чисел: частку і залишок.

Приклад

```
x = 23
y = 7
print(divmod(x, y))    # (3, 2)
```

Функція `hex(x)`

Конвертує ціле число у рядок із шістнадцяткових цифр.

Приклад

```
x = 15
print(hex(x))    # '0xf'
```

Функція `oct(x)`

Конвертує ціле число у рядок із вісімкових цифр.

Приклад

```
x = 15
print(oct(x))    # '0o17'
```

Функція `round(x)`

Округлює число до найближчого цілого.

Приклад

```
print(round(3.0254))    # 3
```

Функція `round(x, n)`

Округлює число `x` з точністю до `n` знаків після коми.

Приклад

```
print(round(3.0254, 2))    # 3.03
```

1.7 Оператори інкременту та особливості `float`

У Python відсутні оператори `++` та `--`. Замість них використовуються комбіновані оператори присвоєння:

- ☑ `x += 1` замість `x = x + 1`
- ☑ `x -= 1` замість `x = x - 1`
- ☑ `x *= 2` замість `x = x * 2`

Обережно з дійсними числами (float)!

Числа з плаваючою крапкою зберігаються в пам'яті комп'ютера у двійковій системі з обмеженою точністю (53 біти). Через це обчислення можуть містити незначну похибку:
0.1 + 0.1 + 0.1 буде дорівнювати 0.30000000000000004
Для порівняння дійсних чисел завжди використовуйте функцію `math.isclose(a, b)` або порівняння з точністю ε: `abs(a - b) < 1e-9`.

Контрольні питання до глави 3

1. Що таке змінна у Python? Як їй надається значення?
2. Що таке ідентифікатор? Які правила його побудови?

3. Які символи та слова заборонено використовувати як ідентифікатори?
4. Що таке зарезервовані слова у Python? Наведіть приклади.
5. Що станеться, якщо використати зарезервоване слово як назву змінної?
6. Дайте означення виразу, операнда та оператора.
7. Які правила форматування коду у Python ви знаєте?
8. Як у Python позначають коментарі? Чим відрізняється однорядковий від багаторядкового коментаря?
9. Назвіть основні вбудовані типи даних у Python.
10. Які числові типи даних підтримує Python?
11. Як у Python записуються цілі, дійсні, восьмирічні, шістнадцятирічні та двійкові числа?
12. Як створюються комплексні числа у Python? Наведіть приклад.
13. Які основні арифметичні оператори підтримує Python? Наведіть приклади їх використання.
14. Що означає оператор цілочисельного ділення та оператор остачі від ділення?
15. Які вбудовані функції для роботи з числами ви знаєте? Наведіть приклади їх використання.
16. Як працює функція `round()`? Чим відрізняється `round(x)` від `round(x, n)`?
17. Як можна виділити окремі цифри з двоцифрового чи трицифрового числа?

Практичні завдання до глави 3

1. Створіть змінну `x` та надайте їй значення 25. Виведіть її на екран.
2. Складіть програму, яка перевіряє, чи є число парним або непарним.
3. Напишіть програму, яка виділяє десятки та одиниці з двоцифрового числа.
4. Напишіть програму, яка виділяє сотні, десятки та одиниці з трицифрового числа.
5. Обчисліть результат виразу: `(15 // 4) + (15 % 4)`. Поясніть отриманий результат.
6. Створіть дві змінні `z1` та `z2` як комплексні числа. Виконайте їх додавання та виведіть результат.
7. Використайте функцію `abs()` для знаходження модуля від'ємного числа.
8. Використайте функцію `divmod()` для ділення числа 29 на 5. Виведіть частку та остачу.

9. Використайте функцію `round()` для округлення числа 3.14159 до двох знаків після коми.
10. Напишіть програму, яка обчислює кількість цифр у числі 2^{1000}.

Оператор циклу `for` та функція `range()`

2

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із заздалегідь відомою кількістю повторень є цикл `for`.

```
Синтаксис циклу for
for параметр in range(start, stop, step):
    блок_операторів
```

2.1 Генератор послідовностей `range()`

Найчастіше для формування послідовностей чисел у циклі `for` використовується вбудована функція `range()`. Вона може приймати від 1 до 3 аргументів:

- ☑ `range(stop)` — генерує числа від 0 до $stop - 1$ із кроком 1.
- ☑ `range(start, stop)` — генерує числа від $start$ до $stop - 1$ із кроком 1.
- ☑ `range(start, stop, step)` — генерує числа від $start$ до $stop - 1$ із кроком $step$.

tabularx

Виклик `range()`	**Згенерована послідовність**	**Примітка**
`range(5)`	`0, 1, 2, 3, 4`	Від 0 до 4
`range(1, 6)`	`1, 2, 3, 4, 5`	Від 1 до 5
`range(2, 10, 2)`	`2, 4, 6, 8`	Парні числа від 2 до 8
`range(10, 0, -1)`	`10, 9, 8, 7, 6, 5, 4, 3, 2, 1`	Зворотний відлік

Таблиця 2.1: Приклади роботи функції `range()`

Якщо передати в range() лише одне число, за замовчуванням відлік починається з 0, а верхня межа не включається

```
for i in range(5):
    print(i)
```

Результат виведення : 0, 1, 2, 3, 4

range(start, stop) починає з числа start (включно) і зупиняється перед stop (не включаючи його).

```
for i in range(1,5):
    print(i)
```

Результат виведення: 1, 2, 3, 4

Третій параметр — це крок (step). range(1, 10, 2) починає з 1 і додає 2 на кожному кроці, генеруючи непарні числа до 10

```
for i in range(1,10,2):
    print(i)
```

Результат виведення: 1, 3, 5, 7, 9

Якщо крок від'ємний (-1), значення будуть зменшуватися. Цикл починається з 10 і зупиняється перед 0.

```
for i in range(10,0,-1):
    print(i)
```

Результат виведення: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

2.2 Базові алгоритми з циклом `for`

Приклад 1. Обчислення суми натуральних чисел від 1 до n

```
n = int(input("Введіть n "))
s = 0
for i in range(1, n + 1):
    s += i

print(f"Сума чисел від 1 до {n} дорівнює: {s}")
```

input(...) запитує у користувача число та приймає його як рядок (текст).

int(...) перетворює введений текст у ціле число та зберігає його в змінну n.

$s = 0$ створюється змінна-накопичувач s із початковим значенням 0. У ній буде зберігатися підсумкова сума.

for i in range(1, n + 1): цикл for перебирає числа по порядку.

range(1, n + 1) генерує послідовність від 1 до n включно (верхня межа $n + 1$ не включається в діапазон, тому цикл зупиняється саме на n).

s += i на кожній ітерації циклу поточне число i додається до суми s (це скорочений запис для $s = s + i$).

print(f"Сума чисел від 1 до n дорівнює: s")

За допомогою f-рядка на екран виводиться підсумковий результат.

Приклад 2. Обчислення факторіала n!

```
n = int(input("Введіть n "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print(f"{n}! = {fact}")
```

Приклад 3. Табулювання функції y = sin(x) на відрізку [a, b]

```
import math

a = float(input("Введіть початок відрізка a "))
b = float(input("Введіть кінець відрізка b "))
n = 10
h = (b - a) / n

print(" x | sin(x)")
print("-------------")
for i in range(n + 1):
    x = a + i * h
    y = math.sin(x)
    print(f"{x:6.2f} | {y:6.4f}")
```

Форматоване виведення (виділений рядок):

$\{x : 6.2f\}$ — виводить значення x: занімає 6 символів у ширину та округлюється до 2 знаків після коми.| — виводить розділювальну риску між стовпчиками.

$\{y : 6.4f\}$ — виводить значення y ($\sin(x)$): шириною в 6 символів та округлене до 4 знаків після коми.

Аналіз дільників числа та перевірка на простоту

Математична довідка

Просте число — це натуральне число, більше за 1, яке має точно два різні натуральні дільники: 1 та саме це число. Якщо число n має дільник, відмінний від 1 та n, то найменший такий дільник не перевищує $\lfloor\sqrt{n}\rfloor$.

Приклад 4. Виведення всіх дільників числа

```
n = int(input("Введіть натуральне число n "))
count=0
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
    count=count+1
print(f"Кількість дільників: {count}")
```

Пошук дільників: Програма перебирає всі цілі числа i від 1 до n включно і перевіряє умову n

Виведення та лічильник: Якщо число i є дільником:воно одразу виводиться на екран (print(i)); лічильник count збільшується на 1. Наприкінці програма виводить загальну кількість знайдених дільників.

Обчислення нескінченних математичних рядів

Обчислимо суму $S = \sum_{n=0}^{N} x^n = 1 + x + x^2 + \cdots + x^N$ ($|x| < 1$).

Приклад 5. Сума геометричного ряду

```
x = float(input("Введіть x (|x| < 1) "))
n = int(input("Введіть кількість членів N "))

s = 0.0
term = 1.0

for i in range(n):
    s += term
    term *= x

print(f"Сума ряду: {s:.6f}")
print(f"Точна сума 1/(1-x): {1 / (1 - x):.6f}")
```

Обчислення експоненти e^x через ряд Тейлора

Приклад 6. Обчислення e^x

```
import math

x = float(input("Введіть x "))
n = int(input("Кількість членів ряду "))

s = 0.0
term = 1.0

for i in range(1, n + 1):
    s += term
    term *= x / i

print(f"Наближене значення e^{x} = {s:.8f}")
print(f"Точне значення (math.exp): {math.exp(x):.8f}")
```

f"..."(f-рядок) — дозволяє вставляти значення змінних безпосередньо всередину тексту за допомогою фігурних дужок .

$\{s : .8f\}$ — бере значення змінної s (суму ряду) та форматує його як число з плаваючою крапкою, залишаючи рівно 8 знаків після коми.

Цей рядок показує наближене значення експоненти e^x , вираховане через суму перших n членів ряду Тейлора.

Приклад 7. Таблиця множення від 2 до 10

```
print("Таблиця множення від 2 до 10")
print("====================================")

# Заголовок
for i in range(2, 11):
    print(f"{i:4}", end="")
print()
print("------------------------------------")

# Основна частина таблиці
for i in range(2, 11):          # зовнішній цикл — рядки
    for j in range(2, 11):      # внутрішній цикл — стовпчики
        print(f"{i*j:4}", end="")
    print()
```

Пояснення роботи вкладених циклів на прикладі таблиці множення

Зовнішній цикл (`for i in range(2, 11):`) відповідає за формування рядків таблиці. На кожній ітерації він бере нове значення i (від 2 до 10) і запускає внутрішній цикл.

Внутрішній цикл (`for j in range(2, 11):`) відповідає за стовпчики таблиці. Для кожного значення j (від 2 до 10) він обчислює добуток $i \cdot j$ і виводить його на екран.

Комбінація циклів:

- ☑ Коли внутрішній цикл завершує всі ітерації для певного i, виконується `print()` без аргументів — це перенос рядка.
- ☑ Потім зовнішній цикл переходить до наступного значення i, і знову запускається внутрішній цикл.
- ☑ Таким чином формується таблиця: кожен рядок відповідає одному множнику i, а кожен стовпчик — множнику j.

Приклад роботи:

- ☑ При $i = 2$ внутрішній цикл обчислює: $2 \cdot 2, 2 \cdot 3, \ldots, 2 \cdot 10$.
- ☑ При $i = 3$ внутрішній цикл обчислює: $3 \cdot 2, 3 \cdot 3, \ldots, 3 \cdot 10$.
- ☑ І так далі до $i = 10$.

У результаті отримуємо повну таблицю множення, де **зовнішній цикл керує рядками**, а **внутрішній — стовпчиками**.

2.3 Управління циклом: `break`, `continue` та `else`

- ☑ `break` — достроково припиняє виконання циклу.
- ☑ `continue` — пропускає залишок тіла циклу та переходить до наступної ітерації.
- ☑ `else` — блок коду, який виконується після завершення циклу, якщо вихід з нього **не** відбувся за допомогою `break`.

Приклад 8. Пошук першого від'ємного числа з використанням break та else

```
for i in range(5):
    val = int(input("Введіть число "))
    if val < 0:
        print(f"Знайдено перше від'ємне число: {val}")
        break
else:
    print("Усі введені числа є невід'ємними.")
```

Контрольні питання до теми

1. Який синтаксис циклу `for` у Python?
2. Які параметри може приймати функція `range()`? Поясніть їх призначення.
3. Що відбувається, якщо у `range()` передати лише один аргумент?
4. Як працює `range(start, stop)`? Які межі включаються, а які — ні?
5. Як використати третій параметр `step` у функції `range()`?
6. Що відбувається, якщо параметр `step` є від'ємним?
7. Як за допомогою циклу `for` обчислити суму чисел від 1 до n?
8. Як за допомогою циклу `for` обчислити факторіал числа n?
9. Як можна використати цикл `for` для табулювання функції (наприклад, $y = \sin(x)$)?
10. Як працюють вкладені цикли? Наведіть приклад побудови таблиці множення.
11. У чому полягає різниця між зовнішнім та внутрішнім циклом у вкладеній структурі?
12. Як працюють оператори `break`, `continue` та `else` у циклі `for`?

Практичні завдання до теми

1. Надрукувати числа від 0 до 9 у рядок через пробіл.
2. Надрукувати числа від 0 до 9 п'ять разів.
3. Надрукувати матрицю (таблицю) 5×5, де в першому рядку нулі, у другому — одиниці, і так далі. Потім переписати код так, щоб кількість рядків та стовпців вводив користувач.
4. Надрукувати таблицю множення від 2 до 10.

5. Надрукувати матрицю 5×5, елементи якої — зірочки, а при $i = j$ надрукувати символ \.
6. Запитати користувача номер рядка r та стовпця c, у якому буде символ `*`, всі інші елементи — нулі.
7. Обчислити суму чисел від 1 до n, використовуючи цикл `for`.
8. Обчислити факторіал числа n.
9. Табулювати функцію $y = \sin(x)$ на відрізку $[a, b]$ з кроком h.
10. Вивести всі дільники числа n та їх кількість.
11. Обчислити суму перших N членів геометричного ряду $1 + x + x^2 + \dots$ при $|x| < 1$.
12. Обчислити наближене значення e^x за допомогою ряду Тейлора.